Teste 01 - Exibição do estado inicial

```
================================================================================
SIMULAÇÃO DO ALGORITMO DO BANQUEIRO
Referência: Tanenbaum & Bos - Sistemas Operacionais Modernos
ALUNO: LUIS FERNANDO SOARES SALES
================================================================================


================================================================================
ALUNO: LUIS FERNANDO SOARES SALES
TESTE: TESTE 1 - ESTADO INICIAL
================================================================================
================================================================================
ESTADO ATUAL DO SISTEMA
ALUNO: LUIS FERNANDO SOARES SALES
================================================================================
Recursos Totais (E):      [10, 5, 7]
Recursos Disponíveis (A): [3, 3, 2]
--------------------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
--------------------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [2, 0, 0]       | [3, 2, 2]       | [1, 2, 2]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
================================================================================
```

Explicação do resultado:

- **O sistema possui dez unidades de R0, cinco de R1 e sete de R2.**
- **Após descontar os recursos já alocados, restam [3, 3, 2] unidades disponíveis.**
- **O processo P1, por exemplo, já possui [2, 0, 0] e ainda precisa de [1, 2, 2] para atingir seu máximo declarado.**

Teste 2 — Verificação do estado seguro inicial

```
================================================================================
ALUNO: LUIS FERNANDO SOARES SALES
TESTE: TESTE 2 - VERIFICAÇÃO DE ESTADO SEGURO
================================================================================

[VERIFICAÇÃO DE ESTADO SEGURO]
Aluno: LUIS FERNANDO SOARES SALES
Vetor de Trabalho Inicial (Work): [3, 3, 2]

Passo 1: P1 pode ser atendido!
  Necessidade [1, 2, 2] <= Disponível [3, 3, 2]
  P1 executa e libera seus recursos: [2, 0, 0]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

SUCESSO: O sistema está em um ESTADO SEGURO!
Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4
ALUNO: LUIS FERNANDO SOARES SALES

================================================================================
```

Explicação do resultado:

O algoritmo encontrou uma sequência em que todos os cinco processos podem terminar. Inicialmente, P1 consegue executar com os recursos disponíveis. Ao terminar, libera seus recursos, permitindo que P3 execute. O mesmo raciocínio é aplicado aos demais processos.

A sequência encontrada é:

$$P1 \rightarrow P3 \rightarrow P2 \rightarrow P4 \rightarrow P0$$

Portanto, o sistema começa em um estado seguro. Isso não significa que um deadlock nunca poderá ocorrer, mas que existe pelo menos uma sequência possível de conclusão dos processos para o estado analisado.

Teste 3 — Requisição válida e segura pelo processo P1

```
================================================================
ALUNO: LUIS FERNANDO SOARES SALES
TESTE: TESTE 3 - REQUISIÇÃO SEGURA DE P1
================================================================

################################################################
ALUNO: LUIS FERNANDO SOARES SALES
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P1: [1, 0, 2]
################################################################
TENTATIVA PROVISÓRIA: Recursos disponíveis. Testando a alocação...

[VERIFICAÇÃO DE ESTADO SEGURO]
Aluno: LUIS FERNANDO SOARES SALES
Vetor de Trabalho Inicial (Work): [2, 3, 0]

Passo 1: P1 pode ser atendido!
  Necessidade [0, 2, 0] <= Disponível [2, 3, 0]
  P1 executa e libera seus recursos: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

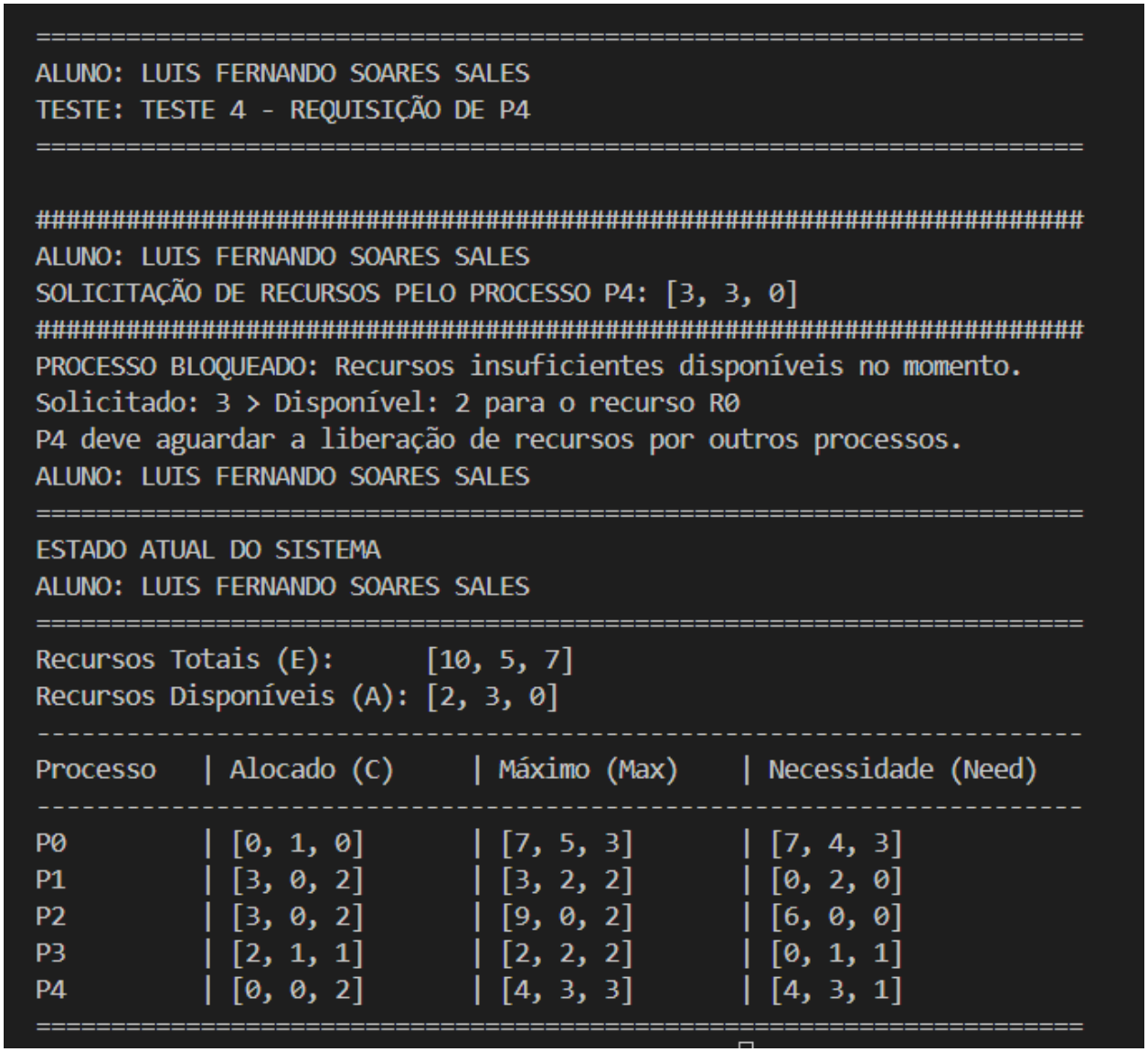
Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]
```

Explicação do resultado:

A solicitação foi concedida porque havia recursos suficientes e, após a alocação provisória, o sistema continuou seguro. O algoritmo encontrou novamente uma sequência que permite a conclusão de todos os processos.

Após a concessão, os recursos disponíveis passam de [3, 3, 2] para [2, 3, 0]. A necessidade restante de P1 passa de [1, 2, 2] para [0, 2, 0].

Teste 4 — Requisição negada pelo processo P4



Explicação do resultado:

A solicitação foi negada porque não há unidades suficientes de R0 disponíveis. O algoritmo identifica o problema antes de tentar alocar os recursos.

Por isso, os recursos disponíveis permanecem [2, 3, 0] e as matrizes de alocação e necessidade não sofrem alterações. O programa não precisa executar o teste de segurança nem realizar um rollback nesse caso.